Yelin Cheon

Aruna Balasubramanian

CSE 310

07 December 2023

Using Multiple GPUs To Train Model

Introduction

With the advancement of Artificial Neural Networks in areas like NLP and Computer Vision, numerous models have been developed. GPUs are employed to train these Artificial Neural Network models. As models become more complex and datasets grow larger, issues such as GPU memory limitations and extended training times arise. To address these problems, complex models like GPT-3 utilize multiple GPUs for training. The method of using multiple GPUs for distributed training is referred to as Distributed Data Parallel (DDP). In distributed training with multiple GPUs, data is exchanged between GPUs. The purpose of this project is to verify whether the problems that occur when using a single GPU are resolved by utilizing DDP, and to understand and examine the communication between GPUs.

Problem Statement

As a model becomes more complex or the number of its parameters increases, the amount of data that the GPU needs to process also increases. This can lead to GPU memory issues. In my other research, I experienced GPU memory overflow problems while training an image

reconstructed model for fine-tuning. At that time, I was using a single NVIDIA GeForce RTX 2080 SUPER GPU, and my MLP (Multilayer Perceptron) model had 8 hidden layers with 512 neurons each. Below is a photo illustrating the GPU memory issue I encountered.

```
    return torch.batch_norm(
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 314.00 MiB (GPU 1; 7.79 GiB total capacity; 7.44 GiB al
ready allocated; 103.44 MiB free; 7.53 GiB reserved in total by PyTorch) If reserved memory is >> allocated memory try set
ting max_split_size_mb to avoid fragmentation.  See documentation for Memory Management and PYTORCH_CUDA_ALLOC_CONF
```

Figure 1. GPU Memory Problem

Approach

To address the problem mentioned in the Problem Statement, instead of training a model with a single GPU, multiple GPUs are used for distributed training. This type of distributed training, as mentioned in the introduction, is referred to as Distributed Data Parallel (DDP). DDP involves copying the entire model to each GPU to train, and then combining the results through inter-GPU communication. To elaborate, each GPU contains the same complete model and the dataset is split and sent in parallel to each GPU. Then, each GPU computes the gradients with its portion of the dataset. Next, these results are sent to a master GPU, which calculates the average gradient and uses it for optimization. This process involves communication between GPUs. Typically, GPU communication uses NCCL (NVIDIA Collective Communication Library), but it is also possible through a TCP connection. In summary, DDP refers to distributed training using inter-GPU communication.

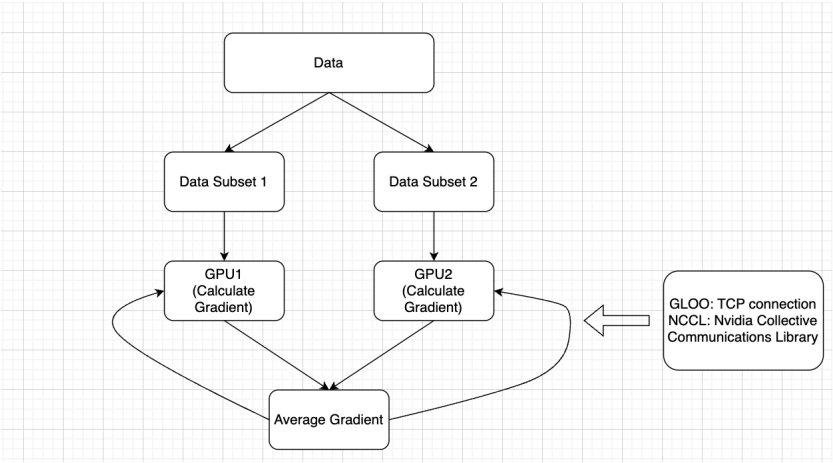

Figure 2. Distributed Data Parallel

Solution

In this project, I used DDP (Distributed Data Parallel) with multiple GPUs to resolve the issues encountered with a single GPU. I conducted experiments using two GPUs. Firstly, I tested if the GPU memory problem mentioned in the Problem Statement was resolved using DDP. Secondly, I compared the training time between using multiple GPUs and a single GPU to determine if DDP actually reduces training time. Thirdly, the training time and performance of a ViT model (Vision Transformer) were compared between single and multiple GPUs. Performance was measured in terms of accuracy. The reason for choosing the ViT model is that transformers, commonly used in the NLP field where DDP is prevalent, also parallelize data in their attention heads, making this model an interesting choice for experimentation over a standard MLP model. This experiment allows for a comparison of performance and training time in a model frequently used in DDP. In the fourth experiment, I compared the performance and training time based on different GPU communication methods, using the ViT model and

measuring performance in terms of accuracy. The communication methods compared were NCCL and TCP(GLOO). This experiment aims to determine which communication method is more effective in DDP. The fifth experiment observed the data transmission volume when using a TCP connection for GPU communication in the ViT model. It allowed for observation of data transmission in forward, backward, and total gradient processes, depending on the batch size.

Evaluation Set Up

1	Check if the memory error occurs with a deep and wide model or not.
2	Compare the performance and train time between single GPU and Multi GPUs.
3	Compare the performance and train time with a small vit model between a single GPU and multi GPUs.
4	Compare the performance between tcp connection and NCCL.
5	Observe transported data byte.

Table 1. Experiment list

For the first three experiments, the parameter is the number of GPUs, and DDP was used when employing double GPUs. The fourth experiment's parameter is the method of communication between GPUs, comparing GLOO (TCP) and NCCL. The fifth experiment's parameter is the batch size. I used the MNIST dataset, with 60,000 training samples and 10,000 test samples. The batch sizes used were 32, 64, and 128. My anticipated results for each experiment are as follows. First, I expect the GPU memory issue to be resolved in the first experiment because the data is distributed across GPUs, reducing the load on each one. In the

second experiment, I expect the training time with DDP to be faster than with a single GPU, as each GPU trains simultaneously. For the third experiment, I anticipate similar performance between single and multi-GPU setups, with multi-GPU being faster in training time. This is because the hyperparameters remain the same, but DDP allows for faster training. Particularly with the ViT model, I expect the DDP training time to be significantly faster due to data division and parallel processing within the model. In the fourth experiment, I expect similar accuracy with NCCL and GLOO (TCP), but faster training time with NCCL due to overhead in TCP. In the fifth experiment, I anticipate more data transmission during forward passes than backward, and a decrease in data transmission with larger batch sizes, as the ViT model is heavier in forward passes and larger batches reduce the amount of transported data processed at once in the ViT model. For visualizing and analyzing the results, I used heatmaps, tables, and graphs. Heatmaps were effective for comparing PSNR and training time through color coding. The third and fourth experiments' training times were represented using bar graphs, and accuracy using linear graphs. I emphasized precise numerical results by using tables.

Result & Analysis

Experiment 1

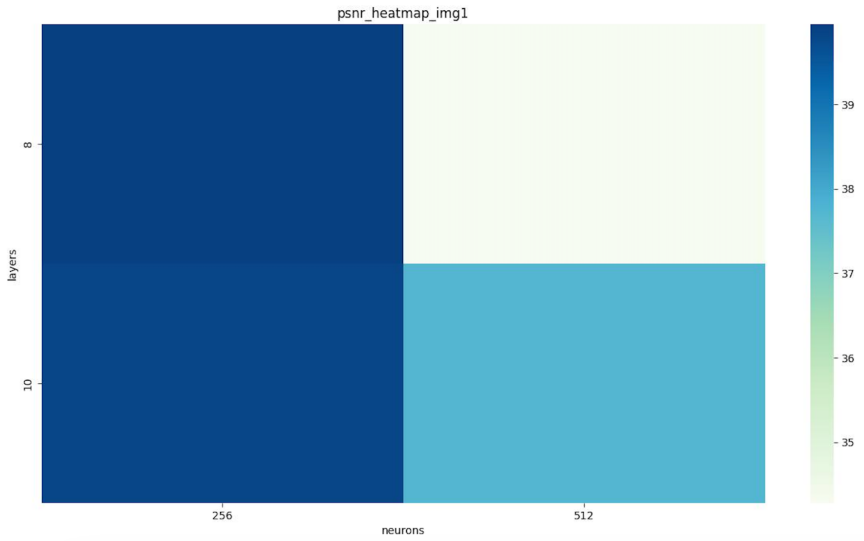

Figure 3. PSNR Heatmap

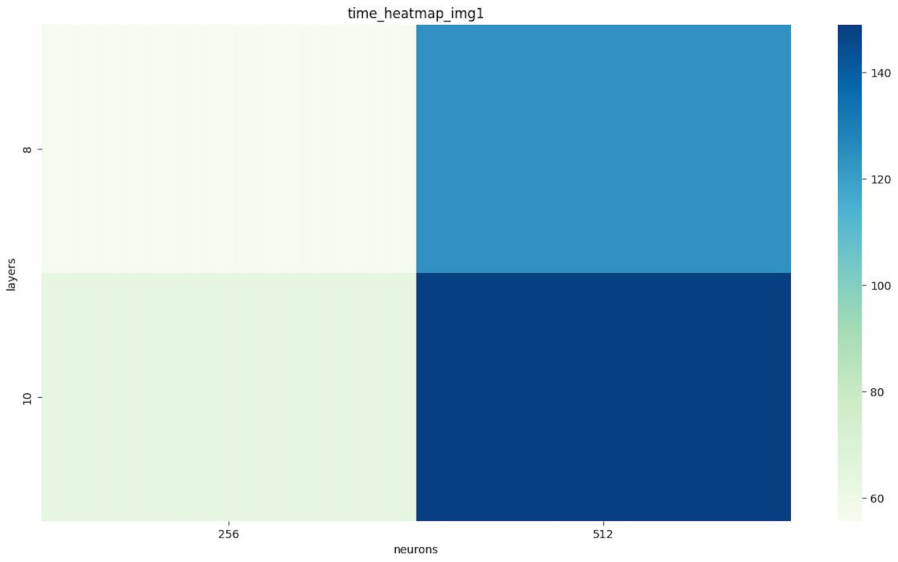

Figure 4. Time Heatmap

Experiment 1 showed that the GPU memory issue encountered when using a single GPU was resolved. My hypothesis for Experiment 1 was correct, demonstrating that DDP can solve GPU memory problems. However, as can be seen in the heatmap, it is evident that overfitting occurred with 512 neurons.

Experiment 2

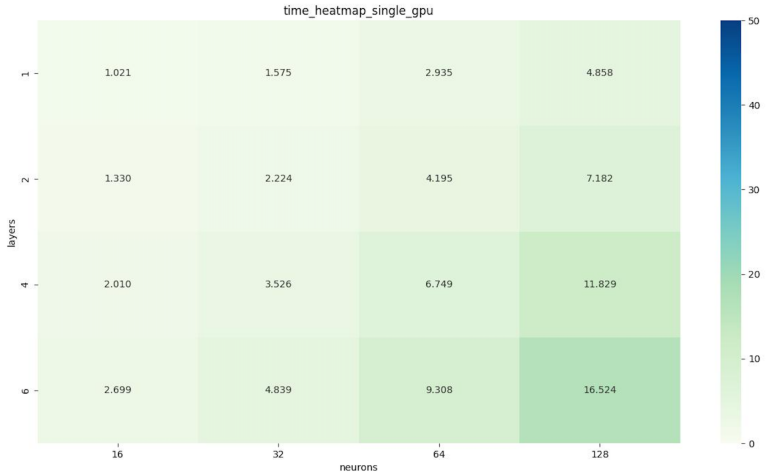

Figure 5. Time Heatmap with single GPU

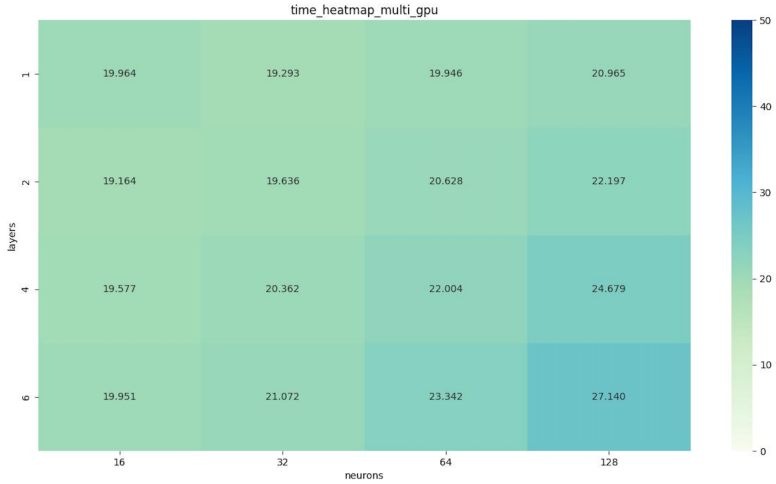

Figure 6. Time Heatmap with multi GPUs

According to the heatmap, training with a single GPU was faster than using multiple GPUs. This was observed while using a model for image reconstruction, indicating that for basic MLP models, a single GPU can be faster. The reason for this outcome might be due to the time taken for data exchange in multi-GPU setups through inter-GPU communication. Additionally, the use of TCP connection could have contributed to this result. Typically, TCP connections are not often used for GPU communication due to overhead issues. Furthermore, the amount of data I used was small, and it becomes evident that using a single GPU is more efficient for such a small dataset and a simple model. Thus, my hypothesis for the second experiment, which anticipated faster training times with multiple GPUs, was proven incorrect.

Experiment 3

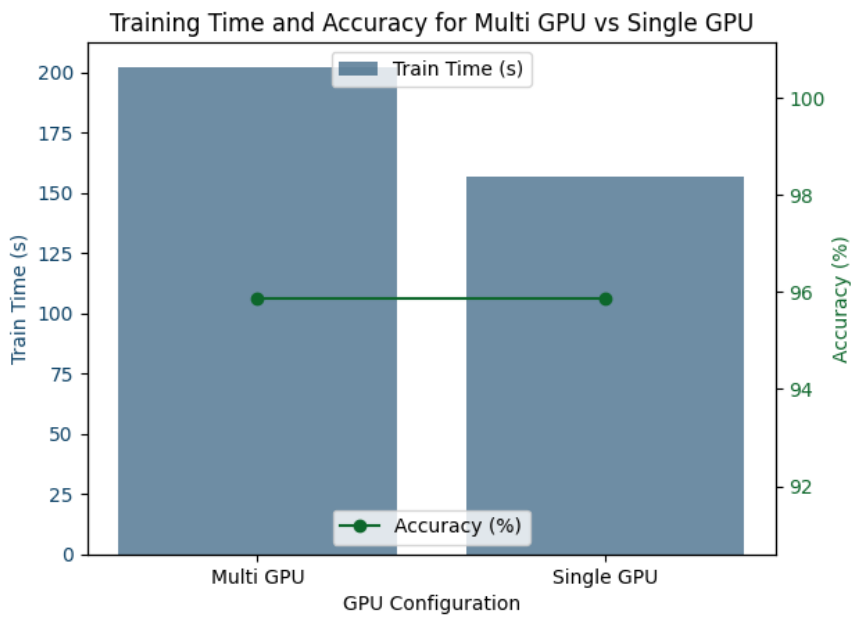

Figure 7. Performance and Training time between Multi GPUs and a Single GPU

As can be seen in the graph, while my expectation that the performance would be similar was correct, my prediction that the training time would be slower with a single GPU was incorrect. This, like in Experiment 2, is thought to be due to the time incurred for GPU communication and the overhead caused by using a TCP connection.

Experiment 4

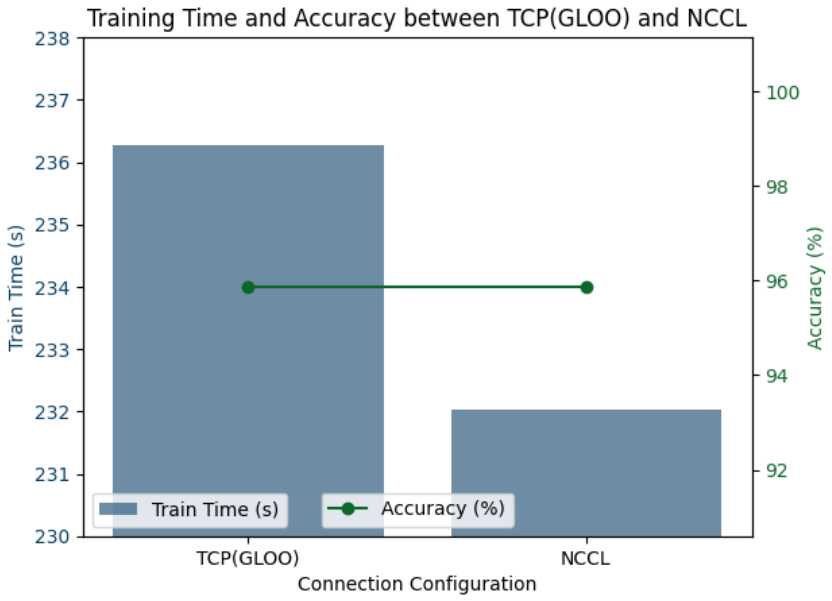

Figure 8. Performance and Training time between TCP and NCCL

As evident from the graph, my hypothesis for Experiment 4 that performance would be similar and using NCCL for GPU communication would be faster, was correct. GPU communication often utilizes NCCL, and by using NCCL, it's possible to address issues like the overheads that occur with TCP connections.

Experiment 5

	Forward	Backward	Total Gradient (All-reduce) process
batch size = 32	188160000 bytes	7500 bytes	39915000 bytes
batch size = 64	188160000 bytes	3752 bytes	19968144 bytes
batch size = 128	188160000 bytes	1876 bytes	9984072 bytes

Table 2. Comparison of transported data volume depending on batch size

The table showing the results of Experiment 5 confirms my hypothesis that the data transmission volume during forward propagation would be greater than during backward propagation. First, the data transmission volume in forward propagation was overwhelmingly higher than in backward propagation. I believe this is due to the characteristics of the ViT model. In this experiment, I split 28 * 28 MNIST images into 49 pieces and sent them in parallel to the attention heads. Since the image is fragmented and the data is sent in parallel, it is inevitable that ViT has a higher data transmission volume in forward than in backward. However, the variation in data transmission volume with batch size in Experiment 5 shows that my hypothesis was incorrect. In forward propagation, there was no change in data transmission volume with different batch sizes, possibly because the weights are shared equally in DDP. For other processes besides forward, it was observed that the data transmission volume decreases as the batch size increases, confirming my expectation. This indicates that as the batch size increases, more data is processed at once, resulting in a reduction in data transmission volume.

Conclusion

	Comparison
deep and wide model	Single GPU : X | Multi GPUs : O
train time with tfd model	Single GPU < Multi GPU
Performance	Same
train time with vit model	Single GPU < Multi GPU

Table 3. Comparison of single GPU and multi GPUs

	Comparison TCP(GLOO) / NCLL
Performance	Same
train time	TCP(GLOO) > NCCL

Table 4. Comparison of TCP and NCCL

In conclusion, DDP can resolve GPU memory issues that may arise in single GPU setups, especially with wide and deep models. However, in cases of small datasets and simple models, training time was faster with a single GPU, indicating that using a single GPU is more efficient in such scenarios. Additionally, there was no difference in performance between single and multi-GPU setups. In the ViT model, the training time was also faster with a single GPU, which I suspect is due to overhead issues with TCP connections and the time taken for data transmission in GPU communication. When comparing NCCL and TCP, NCCL was faster as it resolved the overhead issues associated with TCP. Finally, there was no change in the amount of data transmission during forward propagation with varying batch sizes, but the data transmission

volume during backward and total gradient processes was inversely proportional to the batch size. This is because, more data is processed at once, the amount of data that needs to be transmitted is reduced.

References

PyTorch. "DistributedDataParallel." PyTorch Documentation.
https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html#torch.nn.parallel.DistributedDataParallel

Telesens. "Distributed Data Parallel Training using PyTorch on AWS." Telesens, 4 April 2019.
https://www.telesens.co/2019/04/04/distributed-data-parallel-training-using-pytorch-on-aws/

Git

https://github.com/ycheon1101/DDP_train/tree/main